

Selective routing problem with synchronization

Jorge Riera-Ledesma^{a,*}, Juan-José Salazar-González^b

^a Departamento de Ingeniería Informática y de Sistemas, Universidad de La Laguna, La Laguna, Spain

^b Departamento de Matemáticas, Estadística e Investigación Operativa, Universidad de La Laguna, La Laguna, Spain

ARTICLE INFO

Keywords:

Routing
Orienteering problem
Synchronization
Branch-and-price-and-cut
Scheduling

ABSTRACT

We introduce a new optimization problem arising in the management of a multi-object spectrograph in a telescope. The field of view of the spectrograph is divided in 55 contiguous and parallel spatial bands. Each band is associated with two opposite sliding bars that can be positioned to observe an astronomical object. The observation of an object requires the synchronized configuration of the bars in several contiguous bands to form a slitlet in the position of the object. A multi-slit pattern for the bars permits the simultaneous observation of several objects requiring bars of different bands. Each astronomer using this spectrograph propose a list of objects to observe. Since the instrument is quite demanded, the manager of the telescope imposes a time limitation to the astronomers, so they need to select a subset of objects from their lists. This paper describes and solves the selection problem. The problem must also find a sequence of configurations for the bars in each band to maximize the total priority of the selection, subject to some synchronization issues and the time limitation. We show three mathematical formulations. One of them is a set-partitioning model where a master problem manages the synchronization constraints and a subproblem generates columns satisfying the time limitation. We propose two branch-and-price-and-cut algorithms for solving the set-partitioning model, and discuss an extensive computational experiment showing the performance of the algorithms on two families of instances.

1. Introduction

The Gran Telescopio Canarias (GTC), also known as GRANTECAN, is presently the largest optical telescope in the world, and one of the Spanish Unique Scientific and Technical Infrastructures (ICTS).¹ It is a 10 m class telescope located at the *Observatorio del Roque de los Muchachos* in the island of La Palma (Canary Islands, Spain). The Regional Government of the Canary Islands, the Spanish Government, European Funds, and also non-European partners (*Instituto de Astronomía de la Universidad Nacional Autónoma de México* and the *University of Florida*, among others) actively support the GTC. A defining aspect of this scientific infrastructure is its ability to deliver cutting-edge data through new instruments.

One of the most recent instruments in GTC is the Near Infrared Multi-Object Spectrograph (EMIR),² operational since 2017. It is a wide-field near-infrared camera-spectrograph imager which is able to observe objects in a spectroscopic view field of 4×6.64 square arcminutes. One of the novel aspects of EMIR is the use of a cryogenic robotic unit that

permits the remote configuration of a multi-slit pattern in its field of view, where the GTC forms the image of the sky. This device is called Cold Slit Unit (CSU) and is composed by 55 pairs of opposite sliding bars that can be moved in one direction to create a slitlet. Thus, the field of view is divided into 55 bands, each one associated with two retractable opposite bars of height h arcseconds. The value h is fixed by the device. The width w of the slitlet created by a pair of bars is selectable by the astronomer according to the observing conditions and to the required spectral resolution. Fig. 1 shows three photos of the CSU in EMIR.

On a regular basis, normally each semester, the interested astronomers around the world submit their observing proposals to a committee at GTC that distributes the available observing time of each instrument according to their scientific merits. EMIR has a large over-subscription factor, which forces the successful proposals to be customarily readjusted by the applicants to be doable within the awarded time slot. This task implies.

* Corresponding author.

E-mail addresses: jriera@ull.es (J. Riera-Ledesma), jjsalaza@ull.es (J.-J. Salazar-González).

¹ <http://www.ciencia.gob.es/portal/site/MICINN/ICTS>.

² <http://www.gtc.iac.es/instruments/emir/>.

- (a) Selecting about three view fields for the EMIR to cover objects in the submitted proposal: each view field is characterized by a telescope pointing (which defines the center of the view field) and the orientation of the CSU; the awarded observing time should be split among these view fields, thus forcing a time limitation on each one.
- (b) Selecting a subset of objects from the proposal to observe in each selected view field within its time limitation: the order of observations affects the reconfiguration time of the CSU.

To help the astronomers performing this task, the EMIR instrument team has designed and made available to the user community a graphical tool called Optimized Slits Positioner (OSP)³. It is a Guided User Interface that can overlay the EMIR view field with the CSU in an astronomical image. The user must locate manually the GTC pointing over the image, rotate the CSU to the best position angle, and set a slit pattern for the bars on top of the desired objects in the view field. While the OSP offers some capabilities to automatize the slit pattern for the bars, the other actions depend solely on the astronomer ability. There is no algorithm in OSP to solve the above task, which is a quite complicated optimization problem, combining a continuous optimization task in (a) with a combinatorial optimization decision in (b). Salazar-González (2021) describes a heuristic approach to solve (a) only. Our paper is the first scientific attempt to describe an exact algorithm to solve (b) for a given view field and time limitation decided in (a). This algorithm could be inserted in OSP, so the astronomer will only need to decide about the view fields and time limitations to maximize the scientific return within the awarded observing time.

In this paper we assume that the field of view in the sky is given, and it contains a set of objects of interest to the astronomer. Considering the time awarded to the astronomer, not all the objects can be observed. The optimization problem in this paper is to select a subset of objects and the sequence of the observations that can be performed within the awarded observation time. Some objects in the field of view may need the slitlet created by the bars of several contiguous bands. The bands allocated to each object and the slit width depend on the characteristics of the object (brightness and density) and the position in the field of view. Each band consists of three horizontal areas as illustrated in Fig. 2: an upper dead zone of length d_1 arcseconds, a central area, and a lower dead zone of length d_2 arcseconds. The values d_1 and d_2 are set by the need of avoiding the non-uniform bar edges in these locations due to the mechanical design of the bars, that have to adjust and slid smoothly over the bottom and top bars adjacent to each one.

An object located in the central area of a band can be observed by the single slit of the two bars of that band. When an object is located at the upper or lower dead zone of a band, for technical reasons, observing this object requires the slit created by the bars of (at least) two contiguous bands. Fig. 2(a) shows the three areas dividing a band, with an object in

the central area. The two opposite bars set up a slit focusing the object. Fig. 2(b) shows two contiguous bands with an object falling in a dead zone. To observe that object, the bars in the two bands must be appropriately configured to generate a slit surrounding the object. As mentioned, some objects could need three or more contiguous bands to be observed.

Since the bars can be independently configured, EMIR can perform many simultaneous observations (up to 55), but the synchronization of the bars needed for each observation is fundamental for an efficient management of the telescope. In addition, moving a bar is quite time consuming, so reconfiguration times in the CSU are long in EMIR. The efficient use of the telescope time imposes a carefully planning to maximise the scientific return in the available and limited observing time.

Given a field of view and a time limitation, this paper deals with the problem of selecting objects from a list and configuring the CSU bars. The list of objects (candidates to be observed) with priorities are also inputs. The aim is to process as many prioritized observations as possible. We call this optimization problem the *Selective Routing Problem with Synchronization (SRPS)* as it has characteristics of the Selective Travelling Salesman Problem, also known as Orienteering Problems (OP) and well known in the context of Vehicle Routing Problems (VRPs) (see e.g. Vansteenwegen et al., 2011).

The remaining of the paper has the following structure. Section 2 gives a definition of this new optimization problem. A small numerical example complements this definition. Section 3 makes a bibliographic contextualization of the SRPS as a synchronization problem and relates it to OPs and other similar problems. Section 4 introduces three Mixed Integer Programming (MIP) models for the SRPS. The first one is a compact MIP model based on variables using three indices. It linearizes the synchronization constraints using large positive coefficients to activate/deactivate the conditional constraints. Our computational experiment shows an unsatisfactory performance of the branch-and-cut algorithm performed by a state-of-the-art MIP solver on this model. For this reason, our paper proposes a second model avoiding the use of the large coefficients, and eliminating invalid solutions with inequalities that exploit the combinatorial structure of the problem. Based on a Dantzig-Wolfe decomposition (Dantzig and Wolfe (1960)) of the second model, the paper introduces a third model of set-partitioning type and with a subproblem managing the time limitation constraint. This third model gives rise to two branch-and-price-and-cut algorithms described in Section 5. Section 6 shows and discusses extensive computational results on two families of benchmark instances. The paper ends with a conclusion section. The reader can find a mathematical connection between the three models in the Appendix.

2. Problem setting

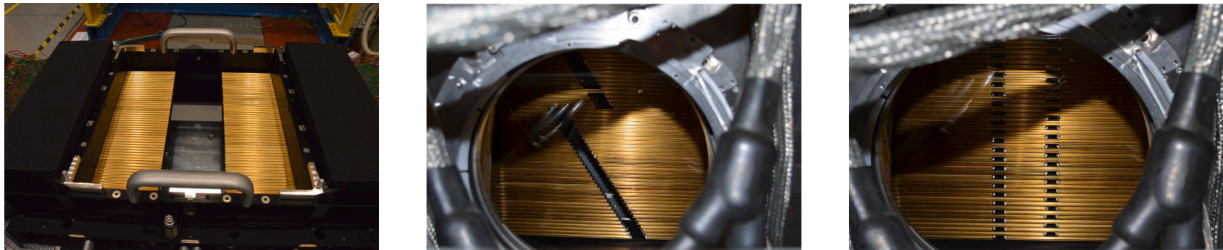


Fig. 1. Three photos of the device with the 55 pairs of bars in different configurations.

SRPS can be seen as a job scheduling problem where the pair of bars of each band represents a configurable processor, and each potential observation represents a job whose processing might require the synchronized configuration of several processors. Using standard

³ <http://www.iac.es/proyecto/emir/pages/observing-with-emir/observing-tools/osp.php>.

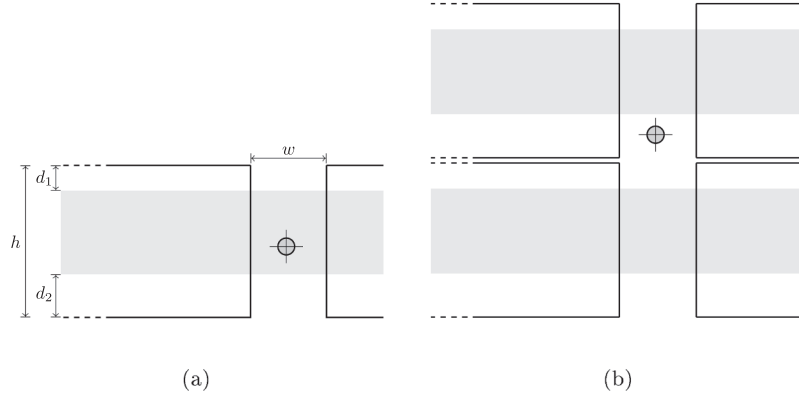


Fig. 2. Details on the bands, slits, and dead zones.

terminology in Job Scheduling, the processors are *non-identical* in the sense that the processors to perform each job are a priori known. Not all jobs must be performed, and each one is associated with a profit. Then, SRPS looks for a selection of jobs and a sequence of configurations for each processor to maximize the sum of the profits of the selected jobs within a time limitation. Performing a subset of jobs consumes setting times of the implied processors, waiting times associated with the synchronization, and processing times for the selected jobs. We now describe the input parameters, characterize a feasible solution, and illustrate the whole problem with a small example.

2.1. Input parameters

Let $J = \{1, \dots, n\}$ be the set of potential jobs. No job in J is compulsorily required to be performed. Processing a job j produces a positive revenue b_j , associated with its priority. Each observation associated with a job $j \in J$ requires a processing time of p_j units. Setting up a processor k from job i to job j consumes a transition time c_{ij} . For convenience of notation, let $t_{ij} = p_i + c_{ij}$. For modeling purposes, we also consider jobs 0 and $n+1$ that are an initial and a final dummy jobs, respectively, and represent a safety status of the processors (in the astronomic application they represent parking the pair of bars of each band in the middle of the field of view). Let $p_j = b_j = 0$ for $j \in \{0, n+1\}$, and $V = J \cup \{0, n+1\}$. Let $K = \{1, \dots, m\}$ be the set of configurable non-identical parallel processors. Each job j requires a subset $K_j \subseteq K$ of processors to be performed. All processors in K_j need to be simultaneously in specific configurations during at least p_j time units to perform j . Similarly, we denote by $J^k \subseteq J$ the set of jobs that require the processor k to be performed. Let $V^k = J^k \cup \{0, n+1\}$.

Let $G^k = (V^k, A^k)$ be the *transition graph* for the processor k , where A^k is the set of arcs among all vertices in V^k , except the ones leaving $n+1$ or entering 0. Let $G = (V, A)$ be the aggregated graph where A is a set of triplets (i, j, k) such that $k \in K$ and $(i, j) \in A^k$. Note that G is a multi-graph since two vertices may be linked by several arcs. Each triplet $a = (i, j, k) \in A$ is associated with a length t_a equal to the time t_{ij} . Finally, let L be the given time limitation to perform the selected jobs. We assume that $L \geq t_{0j} + t_{j,n+1}$ for all $j \in J^k$ and $k \in K$.

2.2. Feasible solutions

We characterize a feasible solution for the SRPS by one elementary path $P^k \subset A^k$ from 0 to $n+1$ in G^k for each processor $k \in K$. The path P^k represents the sequence of jobs that processor k must process starting and finishing at the parking positions. Not all jobs need to be visited by a path, but when a path visits a job j then all paths P^k with $k \in K_j$ must visit j . Also, there must exist a value s_j for each $j \in V$ satisfying:

$$s_i - s_j \leq -t_{ij} \quad k \in K, (i, j) \in P^k \quad (1)$$

$$s_{n+1} - s_0 \leq L. \quad (2)$$

The value s_j represents the starting time to perform j and, together with constraints (1), it guarantees the synchronization of the processors when performing that job. That is, all processors involved in the processing of a specific job j may reach the configuration to start processing j at different times, but they all need to start the process of j at the same time s_j . Inequality (2) limits the overall processing time. The set of inequalities (1) and (2) is known as System of Difference Constraints (see Ahuja et al. (1993)). The SRPS aims at finding the paths P^k and the values s_j maximizing the sum of the profits associated with the selected jobs. From that perspective, SRPS can also be seen as a vehicle routing problem where the jobs are customers and the processors are vehicles.

2.3. An example

This section presents the input parameters and a feasible solution for a small SRPS instance. Fig. 3 represents an input consisting of five potential observations focused by a device with three bands. Objects 1, 4 and 5, located in middle areas, require the bars of exactly one band to be observed. Objects 2 and 3 fall in dead areas, so they need to be observed by the bars of the surrounding bands: the bars of bands 1 and 2 are required to observe the object 2, while the object 3 requires the bars of bands 2 and 3 to be observed. More precisely, in this instance $J = \{1, 2, 3, 4, 5\}$, $K = \{1, 2, 3\}$, $J^1 = \{1, 2\}$, $J^2 = \{2, 3, 5\}$, $J^3 = \{3, 4\}$, $K_1 = \{1\}$, $K_2 = \{1, 2\}$, $K_3 = \{2, 3\}$, $K_4 = \{3\}$ and $K_5 = \{2\}$.

Let us assume that the transition costs c_{ij} are given by the horizontal distance between the objects, with processing times and profit values equal to one unit for all the objects. Fig. 4 depicts all valid transitions for the bars of each band to observe the objects. Each one is the transition graph G^k introduced above, and each arc represents changing the pair of bars of band k from one configuration to another.

When the time limit is $L = 11$, there is a feasible solution for this instance that allows the observation of the five objects. This solution starts setting up the bars of the bands 1, 2 and 3 to observe the objects 1, 5 and 4, respectively. Then, the bars in bands 2 and 3 are set up to

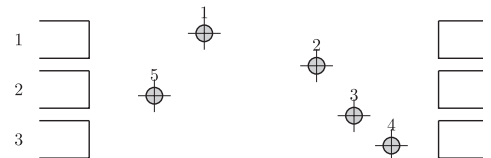


Fig. 3. Example with five objects and three bands.

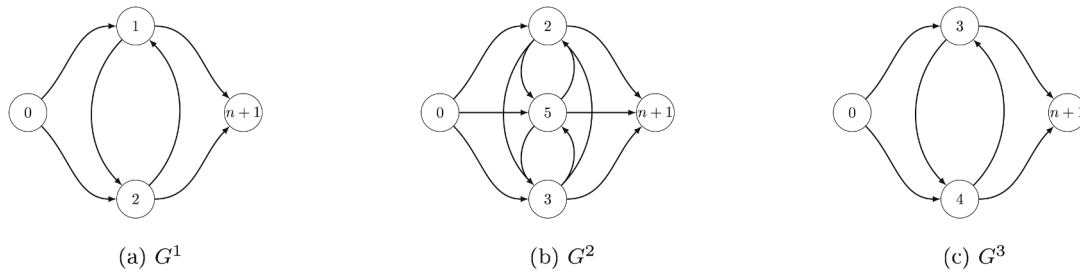


Fig. 4. Transition graphs for the example in Fig. 3.

observe the object 3. Finally, the bars of bands 1 and 2 are configured to observe the object 2. All bars start at configuration 0 and finish at configuration $n + 1$. Fig. 5 shows the elementary paths on the transition graphs.

The vector $s = [0, 1, 8, 6, 4, 2, 11]$ contains valid scheduling time values, according to (1) and (2). Fig. 6 helps to understand the validity of this solution. Observe the existence of idle times for some bars due to the need of synchronization.

3. Literature review

The previous section suggests that SRPS is closely related to vehicle routing problems investigated in the literature. This section details some of these relations.

3.1. Orienteering problems

OPs represent a family of vehicle routing problems taking into account practical situations where serving a customer is optional, generating a profit if the service is done within a time limitation. We refer the reader to the survey by Gunawan et al. (2016) for an extensive revision of this family of problems and their practical applications.

Relaxing the SRPS by removing the synchronization constraints results in m independent instances of the well-known OP (see e.g. Tsiligirides (1984) and Golden et al. (1987)). The OP aims at selecting a subset of customers and designing a route with a total time duration not longer than a given time limitation, to maximize the total collected profit. It is a combination of the *Cycle Problem* and the *Knapsack Problem* where the total time duration of the cycle does not have to be minimized but is constrained.

Although this relaxed version of the SRPS involves a set of vehicles to serve some customers, it should not be confused with the Team Orienteering Problem (TOP) (see e.g. Butt and Cavalier (1994) and Chao et al. (1996)). The TOP is an extension of the OP which also considers multiple routes to serve the customers, but where a customer must be served by one vehicle only. In other words, TOP does not include any synchronization issue and there is not pre-assignment of vehicles to customers. Still, SRPS coincides with TOP when each object requires only one pair of bands to be observed.

3.2. Synchronization problems

According to Drexler (2012), the SRPS is a case of *exact operation synchronization*, “that means that two vehicles must start executing their operations at their respective locations at the same time”. The literature for vehicle routing problems with this characteristic describes exact approaches mainly using arc-variable based formulations making use of time variables representing the beginning of execution for each job. Unfortunately, these formulations are implicitly non-linear, and their linearization requires the use of a big- M value; see e.g. Desrosiers et al. (1995), Lim et al. (2004), Li et al. (2005), Dohn et al. (2009), Cortés et al. (2010) and Gelareh et al. (2013).

An alternative to avoid the big- M drawback is the use of column-generation or branch-and-price approaches. Examples of this technique appear in Ioachim et al. (1999), solving the problems of a fleet assignment and vehicle routing with schedule synchronization constraints, or in Bélanger et al. (2006), approaching the problem of a periodic airline fleet assignment with time windows, spacing constraints, and time-dependent revenues. Both articles use the labeling algorithm proposed by Ioachim et al. (1998).

3.3. Manpower allocation problems

Manpower allocation problems describe an exact operation synchronization family of problems which arises in several contexts where performing each job requires the cooperation between teams, possibly with different skills. The allocation of technicians to service jobs, where solving each specific job requires a combination of technicians with individual skills, is an example of this family of problems. Although these problems share characteristics in common with SRPS like the exact operation synchronization, they also differ in other characteristics like the structure of the routes, the objective functions, and the requirement of processing all jobs.

Lim et al. (2004) introduce the Manpower Allocation Problem as a multi-criteria problem, and proposes a simulating-annealing algorithm to solve it heuristically. Li et al. (2005) cite an application where technicians are dispatched from a central office to various locations in the Port of Singapore to service jobs that may require multiple technicians. Luo et al. (2016) describe an application in the movie industry where some scenes shot at different locations require routing film crews, such as actors, camera operators, lighting technicians, and make-up artists.

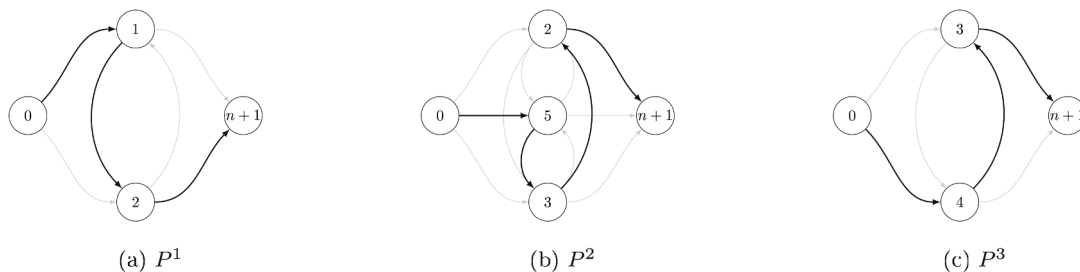


Fig. 5. Elementary paths for the example in Fig. 3.

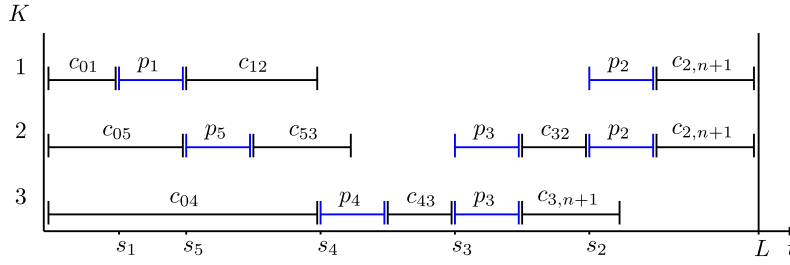


Fig. 6. Diagram associated with a feasible solution for the example in Fig. 3.

Dohn et al. (2009) introduce another application related to airport management: after arriving at the airport, the aircraft stops at different locations and ground crew members of different responsibilities are deployed to deal with various jobs, such as baggage handling, aircraft cleaning, aircraft de-icing service, aviation fueling service, and aircraft maintenance check. The literature also mentions examples arising in the home care sector, where specialized workers travel between the patient homes, which demands collaborative work.

4. Formulation

We now introduce three mathematical formulations to describe the SRPS. They are built on the following two families of variables. For each processor $k \in K$ and each arc $(i, j) \in A^k$, the binary variable x_{ij}^k represents whether the processor k is configured to process the job j immediately after having processed the job i . For each job $j \in J$, the binary variable y_j determines whether the job j is selected to be processed.

Our first model is a compact formulation which also makes use of continuous variables to determine the starting time of each selected observation. It linearizes the time limitation and synchronization constraints (1) and (2) using a big- M value. The second model avoids the big- M value and the continuous variables by eliminating invalid solutions with an exponential number of inequalities on the binary variables x and y . The third model is a set-partitioning formulation based on an exponential number of variables, each one determining a feasible sequence of configurations for a processor.

For the sake of brevity of notation, we define the following standard notation. Given a processor $k \in K$ and a vertex subset $S \subseteq V^k$, we use $\delta_-^k(S) = \{(i, j) \in A^k \mid i \in V^k \setminus S, j \in S\}$ and $\delta_+^k(S) = \{(i, j) \in A^k \mid i \in S, j \in V^k \setminus S\}$. If $S = \{i\}$, we simply write $\delta_-^k(i)$ and $\delta_+^k(i)$. We also use $\delta_-(S) = \{(i, j, k) \in A \mid (i, j) \in \delta_-^k(S), k \in K\}$ and $\delta_+(S) = \{(i, j, k) \in A \mid (i, j) \in \delta_+^k(S), k \in K\}$. We write $x^k(F)$ instead of $\sum_{a \in F} x_a^k$ if $F \subseteq A^k$, and $x(F)$ instead of $\sum_{a \in F} x_a$ if $F \subseteq A$.

4.1. Compact three-index model

As mentioned above, our first model requires continuous variables s_j for each job j in V . The variable s_j represents the starting time to process job j . Then, the following MIP model, say SRPS-1, describes the SRPS:

$$\max \sum_{j \in J} b_j y_j \quad (3)$$

subject to

$$x^k(\delta_+^k(0)) = 1 \quad k \in K \quad (4)$$

$$x^k(\delta_-^k(n+1)) = 1 \quad k \in K \quad (5)$$

$$x^k(\delta_+^k(j)) - x^k(\delta_-^k(j)) = 0 \quad k \in K, j \in J \quad (6)$$

$$x^k(\delta_-^k(j)) = y_j \quad k \in K, j \in J \quad (7)$$

$$s_i - s_j + Lx_{ij}^k \leq L - t_{ij} \quad k \in K, (i, j) \in A^k \quad (8)$$

$$s_{n+1} - s_0 \leq L \quad (9)$$

$$x_{ij}^k \in \{0, 1\} \quad k \in K, (i, j) \in A^k \quad (10)$$

$$y_j \in \{0, 1\} \quad j \in J. \quad (11)$$

The objective function (3) maximizes the profit associated with the selected jobs. Eqs. (4) and (5) establish, respectively, the starting and ending configuration for each processor. Eqs. (6) and (7) determine the variables y_j , and force that each processor $k \in K_j$ must process job j if $y_j = 1$ for each $j \in J$. Constraint (9) is (2), and Constraints (8) linearize constraints (1) since P^k is solution dependent. They have a threefold role: first, they guarantee the connectivity of the configurations for each processors on job j with their respective initial and final parking configurations, 0 and $n+1$; second, they force that each processor $k \in K_j$ is properly configured before start processing a selected job j ; and third, they establish a limitation on the time duration on each processor. Constraints (8) are known in the literature as Miller-Tucker-Zemlin inequalities (see e.g. Gouveia and Pires, 1999). Finally, bounds (10) and (11) establish the limits of the variables x_{ij}^k and y_j , respectively. One could fix s_0 to zero in the formulation and get s_j non-negative, but these constraints do not alter the implicit complexity of the formulation. An alternative compact model is obtained with the non-linear inequalities $x_{ij}^k(s_i + t_{ij} - s_j) \leq 0$ for all $k \in K$ and $(i, j) \in A^k$ instead of (8).

4.2. Combinatorial cuts to manage synchronization

A preliminary computational experiment using a state-of-the-art MIP solver on model SRPS-1 (see Section 6.2) confirmed the expected disappointing performance, even on small instances. This drawback is caused by the use of L in (8), which linearizes constraints (1) and produces a poor Linear Programming (LP) relaxation. For this reason, our next model, say SRPS-2, replaces constraints (8) and (9) by a family of inequalities based on variables x and y , dismissing the big- M drawback.

We generate feasibility cuts studying the combinatorial structure underlying the system (1)–(2). Observe that each binary variable x_{ij}^k triggers a constraint in (1). Indeed, such an inequality is active if there is a triplet (i, j, k) such that $x_{ij}^k = 1$. Given a solution $(\tilde{x}, \tilde{y})$ satisfying (4)–(7), (10), and (11), we define the triplet set $\tilde{A} = \{(i, j, k) \in A : \tilde{x}_{ij}^k = 1\}$. Then $(\tilde{x}, \tilde{y})$ represents a feasible solution for the SRPS if and only if the subsystem defined by $s_i - s_j \leq -t_{ij}$ for all $(i, j, k) \in \tilde{A}$ and (2) admits a solution on the variables s . On the base of this subsystem, we can establish a mechanism to remove an infeasible solution with the inequality

$$x(\tilde{A}) \leq |\tilde{A}| - 1, \quad (12)$$

whenever $\tilde{A}$ determines an infeasible subsystem of (1)–(2). The inequality (12) is dominated by another inequality of the same family if

removing an arc from $\tilde{A}$ determines another infeasible subsystem. In other words, only minimal-inclusion sets $\tilde{A}$ determine non-dominated inequalities, and these sets are called Minimum Infeasible Subsystem (MIS).

Inequality (12) suggests a standard method to approach problems involving synchronization (Luo et al., 2016), time windows (Ascheuer et al., 2001), pickup and delivery (Cordeau, 2006) or loading (Alba Martínez et al., 2013) constraints. However, they are known to be weak cuts in cutting-plane approaches, so it is worthy to characterize the combinatorial structure underlying a MIS, to lift coefficients of the inequalities, and to design efficient procedures to find violated ones. We next characterize the MIS for the SRPS.

4.2.1. Structure of minimum infeasible systems

Assume that we have a set $\tilde{A}$ obtained as a solution of the relaxation of (3)–(7), (10) and (11). We are interested in determining a MISs from $\tilde{A}$. Let us define γ_a as the dual variable associated with each constraint (1) for $a = (i, j, k) \in \tilde{A}$, and define μ as a dual variable associated with the constraint (2). We write $\gamma(F)$ instead of $\sum_{a \in F} \gamma_a$ if $F \subseteq \tilde{A}$ and t_a instead of t_{ij} for $a = (i, j, k) \in \tilde{A}$. Using Duality Theory, $\tilde{A}$ includes an infeasible subsystem if and only if the linear program

$$\min L\mu - \sum_{a \in \tilde{A}} t_a \gamma_a \quad (13)$$

subject to

$$\gamma(\delta_+(j)) - \gamma(\delta_-(j)) = \begin{cases} \mu & \text{if } j = 0 \\ -\mu & \text{if } j = n+1 \\ 0 & \text{otherwise} \end{cases} \quad j \in V \quad (14)$$

$$\gamma_a \geq 0 \quad a \in \tilde{A} \quad (15)$$

$$\mu \geq 0 \quad (16)$$

is unbounded. Observe that a solution (γ, μ) can be interpreted as a flow moving μ units from 0 to $n+1$ in a subgraph of the multigraph G . Constraints (14) guarantee the flow conservation, and inequalities (15) and (16) force non-negativity to the dual variables γ and μ , respectively.

The objective function (13) minimizes the sum of two conflicting components. The first component minimizes the amount of units μ moved from 0 to $n+1$. The second component searches for a maximum-weight flow, assuming t_a to be the weight of arc a . Then, the linear program (13)–(16) is unbounded if, and only if, either $\tilde{A}$ contains a circuit, or $\tilde{A}$ contains a path from 0 to $n+1$ with a length greater than L . This circuit or path induces an infeasible set associated with a violated inequality (12). Any infeasible set describing a simple cycle is a MIS. Any infeasible set describing a path may contain several MISs, each one associated with a subpath of length greater than L .

4.2.2. Strengthening combinatorial cuts

From each circuit or path obtained from $\tilde{A}$, as described above, we can add a violated inequality (12) to the master problem. We now propose a way to strengthen the inequality as follows:

- Let us consider a circuit $C \subseteq \tilde{A}$ processing a subset of jobs $V(C) \subseteq J$. This circuit consists of a sequence $\mathcal{P}(C)$ of paths. Each path P is related to a processor $k(P)$ performing $p(P)$ jobs in $V(C)$. For simplicity of notation, let us denote the sequence of jobs in P by the positions 1, 2, ..., $p(P)$, and define

$$\begin{aligned} \Delta^F(P) &= \{(i, j, k(P)) \in A \mid 1 \leq i < j \leq p(P)\}, \\ \Delta^B(P) &= \{(i, j, k(P)) \in A \mid 2 \leq j < i \leq p(P)\}. \end{aligned}$$

which are the arcs spanning forward and backward, respectively, the

precedence relations between nodes in the path P . Note that the arcs in P are in $\Delta^F(P)$ while $\Delta^F(P)$ has no arc arriving at node 1. Fig. 7(a) and (b) illustrate the notations Δ^F and Δ^B , respectively.

Then the lifted inequality

$$\sum_{P \in \mathcal{P}(C)} x \left(\Delta^F(P) \cup \Delta^B(P) \right) \leq |C| - 1$$

is valid for the SRPS, and avoids a solution processing the job in position 1 before the job in position $p(P)$ for each path P in the circuit C . Indeed, precedences creating a circuit make the synchronization impossible, and therefore the solution infeasible. Similar families of constraints have also been used to strengthen infeasible path constraints for other optimization problems (see e.g. Ascheuer et al. (2001)). The lifted inequality can be further strengthened by the stronger constraints

$$\sum_{P \in \mathcal{P}(C)} x \left(\Delta^F(P) \cup \Delta^B(P) \right) \leq y \left(V(C) \setminus \{i\} \right) \quad \text{for all } i \in V(C) \quad (17)$$

since a sub-circuit processing some jobs in $V(C)$ would also make the solution infeasible. The validity of (17) follows because all jobs are optional and $|V(C) \setminus \{i\}| = |C| - 1$.

- Let us now consider a path $P \subseteq \tilde{A}$ from 0 to $n+1$ to process $p(P)$ jobs with a length greater than L . For $i = 1, \dots, p(P) - 1$ let us define the subpath P_i of P as the path starting from the job in position i , with minimum cardinality, ending with the job $n+1$, and with a length greater than L ; if it exists, the inequality

$$x(\Delta^F(P_i)) \leq |A(P_i)| - 1 \quad (18)$$

avoids solutions including the invalid path P_i .

The huge number of inequalities (17) and (18) requires separation procedures to dynamically detect and insert violated ones when necessary. These procedures are described in Section 5.2.

4.3. Set-partitioning formulation

Forcing the time limitation (1)–(2) with the use of MIS elimination constraints leads in practice to the enumeration of many similar invalid solutions (in a sense, the index k creates symmetries) and therefore to computational inefficiency. We investigated procedures to drop the index k and solve the drawback, but we did not succeed in keeping a compact formulation. We propose an alternative formulation based model from a Dantzig-Wolfe decomposition of the continuous relaxation

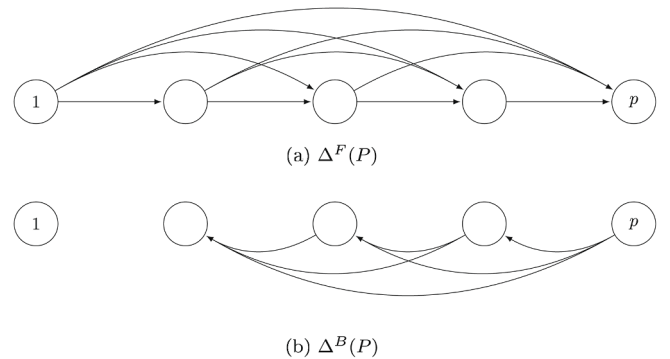


Fig. 7. Strengthening an infeasible-path constraint for a path P .

of SRPS-2, where its subproblem manages the time limitation constraints for each processor.

The new model makes use of a set of mathematical variables representing all possible elementary paths, for each processor $k \in K$, starting at 0 and ending at $n + 1$, with a length not greater than L . However, producing paths that individually fulfill their time limitation does not guarantee a solution globally fulfilling the time limitation constraints. This is caused by the waiting times inherent to the synchronization constraints (see Fig. 6). Thus, although we expect significant computational improvements with a column-generation approach, constraints (18) are still necessary to force the length requirement in a global solution.

Let Ω^k represent all possible elementary paths on G^k , from 0 to $n + 1$, not longer than L . For a given path (or route) $r \in \Omega^k$, for some $k \in K$, let $x_{ij,r}^k$ be a parameter representing the number of times the route r traverses the arc $(i,j) \in A^k$. Note that parameters $x_{ij,r}^k$ are not mathematical variables but a description of r .

We base this new formulation on the following set of mathematical variables. For each $k \in K$ and each $r \in \Omega^k$, a binary variable λ_r^k determines whether processor k in the solution follows the sequence r . We also make use of variables x_{ij}^k and y_i , having the same meaning as in the models SRPS-1 and SRPS-2, and allowing to use valid inequalities like (17) and (18). Then, the SRPS admits a new model SRPS-3 described by (3) and subject to (6), (7), (10), (11), (17), (18) and for each $k \in K$:

$$\sum_{r \in \Omega^k} \lambda_r^k = 1 \quad (19)$$

$$\lambda_r^k \geq 0 \quad r \in \Omega^k \quad (20)$$

$$\sum_{r \in \Omega^k} x_{ij,r}^k \lambda_r^k = x_{ij}^k \quad \left(i, j \right) \in A^k. \quad (21)$$

Constraints (19) and (20) are the convexity constraints, and Eqs. (21) define the coupling of the variables λ with the variables x . Note that there is no need to force λ_r^k to be binary in the formulation because (10) and because an integer array x_{ij}^k corresponds to a unique route $r \in \Omega^k$.

This alternative formulation requires iteratively solving a master problem that introduces columns generated by a subproblem defined on the dual variables of the linear relaxation of the master problem. A branching phase is also necessary to ensure integrability of the solution. The whole approach is described in the next section.

5. Branch-and-price-and-cut-algorithm

Branch-and-price-and-cut (BPC) approaches are branch-and-bound algorithms where computing the dual bounds requires column-generation and row-generation procedures. They are iterative approaches in a decomposition framework where there is a restricted master problem feeded by subproblems generating violated inequalities and variables with negative reduced cost. We herewith describe the main elements of our BPC algorithms to solve the model SRPS-3. These elements are the column-generation procedure (known as “pricing algorithm”), the row-generation procedure (known as “separation algorithm”), and the branching strategy (inherent to any branch-and-bound scheme). We refer the reader to Desrosiers and Lübbecke (2011) for general details on this type of algorithms. We show two BPC implementations. The first one considers the sets Ω^k containing only elementary routes (algorithm BPC-SRPS-E), while the second considers the set Ω^k also including non-elementary routes (algorithm BPC-SRPS-N).

The two implementations make use of the same master problem, as defined in the previous section. This problem is based on the variables x_{ij}^k, y_i and λ_r^k to determine the selection and routing in a solution. The

model also includes the combinatorial cuts (17) and (18) described in Section 4.2 to guarantee the synchronization requirement. These cuts only need the x variables, and can be managed with a branch-and-cut framework. In addition, the SRPS also requires a limitation on the length of each route which is manageable with a dynamic-programming framework inside the subproblem generating the λ_r^k variables. It is worth mentioning that the reduced cost of a λ_r^k variable can be computed using only the dual variables of constraints where they are in the model. This means that only the dual variables of equations (19) and (21) are needed to calculate the reduced cost of a λ_r^k variable. More precisely, for each processor $k \in K$, let us denote by α^k and β_{ij}^k the dual prices associated with constraints (19) and (21), respectively. The reduced cost of a route $r \in \Omega^k$ is

$$\bar{c}_r^k = 0 - \left(\alpha^k + \sum_{(i,j) \in A^k} x_{ij,r}^k \beta_{ij}^k \right).$$

Since the objective function of the master problem maximizes a linear function, the pricing subproblem associated with processor k looks for a route $r \in \Omega^k$ with maximum reduced cost $\bar{c}_r^k$. Thus, the subproblem coincides with computing a minimum-cost route r on the graph G^k where the cost of the arc $(i,j) \in A^k$ is β_{ij}^k . Our approach to solve each subproblem is described in the next section.

5.1. Pricing algorithms

A Sub-Problem (SP) performs a dynamic column-generation approach to overcome the exponential number of variables λ_r^k in the model SRPS-3. This approach leads to an iterative procedure where, at each iteration, new columns enlarge a restricted master problem. The master problem starts solving an initial LP-relaxation with some λ variables. Then, at each iteration, the SP aims at generating routes $r \in \Omega^k$ with negative reduced cost $\bar{c}_r^k$ from the values of the dual variables obtained from the restricted master problem. One route is computed for each processor k , thus up to $|K|$ routes can be generated at each iteration. Each new route r is associated with a new variable λ_r^k for the restricted master problem in the next iteration.

The SP where Ω^k includes only elementary routes is called SRPS-SP-E, and the one where Ω^k also includes non-elementary routes is called SRPS-SP-N. They require implicitly generating all negative reduced cost paths r , from 0 to $n + 1$, with length not greater than L , within each graph G^k , for all $k \in K$. The usual procedure to approach these subproblems is to formulate them as constrained shortest path problems, and solving them with a dynamic programming (DP) approach based on labeling algorithms (see e.g. Irnich and Desaulniers (2005)).

The SRPS-SP-E is the so-called Elementary Shortest Path Problem with Resource Constraints (ESPPRC). Unfortunately, solving SRPS-SP-E using a DP algorithm may become unaffordable as the size of the problem grows because the ESPPRC is strongly $\mathcal{NP}$ -hard (see e.g. Irnich and Villeneuve (2006)). That is why solving a relaxed version of the ESPPRC, as the SRPS-SP-N, becomes an inviting alternative.

Our DP implementation for solving SRPS-SP-N makes use of a relaxed structure called ng -route. The ng -route relaxation, introduced in Baldacci et al. (2011), makes use of the sets $N_i \subseteq V$ for vertex i (according to some criterion), such that $i \in N_i$ and $|N_i| \leq \Delta(N_i)$, where $\Delta(N_i)$ is a parameter (e.g., $\Delta(N_i) = 5$, for all $i \in V$, and N_i contains i and the four nearest customers to i). These sets N_i allow us to associate with each subpath generated during the pricing algorithm with a set of forbidden successors.

This adaptation of the ng -route relaxation solves the pricing problem SRPS-SP-N considering a superset of routes Ω^k from the set of routes Ω^k . The set Ω^k satisfies the feature that none of the routes in $\Omega^k \setminus \Omega^k$ can be used in a feasible integer solution of the master problem. Thus, an

optimal solution of the master problem on Ω^k has the same value as an optimal solution of the problem on Ω^k , but the LP-relaxation from the latter is stronger than that from the former.

5.2. Separation algorithms

We describe now how to dynamically generate the set of MIS elimination constraints (17) and (18) for our BPC algorithms BPC-SRPS-E and BPC-SRPS-N. Let $(\tilde{x}, \tilde{y})$ be a solution from the LP-relaxation of a restricted master problem, also described by its associated multigraph $\tilde{G}$.

If $(\tilde{x}, \tilde{y})$ is an integer solution, we can trivially find circuits in $\tilde{G}$ with the algorithm in Johnson (1975) to separate constraints (17). If there are no cycles in such an integer solution, to separate constraints (18), one can find longest paths with the algorithm in Dijkstra (1959) to compute the shortest path from 0 to $n+1$ in $\tilde{G}$ weighting each arc (i, j, k) with $-t_{ij}$.

If $(\tilde{x}, \tilde{y})$ is a non-integer solution, finding circuits or paths leading to violated inequalities (17) or (18), respectively, is much harder but still affordable. To detect and find cycles we look for minimum-weighted circuits in $\tilde{G}$ assigning a weight $1 - \tilde{x}_{ij}^k$ to each arc (i, j, k) . To detect violating paths longer than L , we look for minimum-weighted paths from 0 to $n+1$ weighting each arc (i, j, k) by $-\tilde{x}_{ij}^k t_{ij}$.

To reduce the complexity when $(\tilde{x}, \tilde{y})$ is not integer, we propose the following heuristic approach: Fix $\mu = 1$, solve the linear program (13)–(16), let $\tilde{y}$ be an optimal solution, and compute $\tilde{G}$ as the graph resulting from removing from G the arcs associated with null values of $\tilde{y}$. If the problem is infeasible, we look for violating cycles in $\tilde{G}$ by using the Johnson's algorithm. If we find a violating cycle, we add the associated inequality (17) to the master problem. Otherwise, we check all paths from 0 to $n+1$ in $\tilde{G}$, evaluating the violation of any potential constraint (18) for each path with length greater than L .

5.3. Branching strategy

When the pricing procedure in Section 5.1 and the separation procedure in Section 5.2 do not find any new element to add to the current restricted master problem, but its solution $(\tilde{x}, \tilde{y})$ contains fractional values, our BPC algorithms resort to the branching phase. In this phase two restricted master problems are created by fixing a variable y_j to 0 and 1, respectively, if $\tilde{y}_j$ is fractional, and by fixing a x_{ij}^k variable if $\tilde{y}_j$ is integer for all $j \in J$ and $\tilde{x}_{ij}^k$ is fractional. When several variables are at fractional values, the closest to 1/2 is selected for the branching; ties with y_j variables are broken by choosing the one with largest priority b_j , and ties with x_{ij}^k variables are broken by choosing the one with largest transition cost c_{ij} .

6. Computational experiments

This section analyzes the performance of the implementations for the exact algorithms BPCSRPS-E and BPC-SRPS-N. It also compares these algorithms against giving the compact model SRPS-1 to IBM ILOG CPLEX 12.10. Although it is well known the weakness of the linear programming relaxation of the compact model, the section shows computational results to precisely measure this negative behaviour, while justifying the need of a more sophisticated alternative approach. For this purpose, we have performed a test campaign on benchmark instances. The description of the test set and a discussion on the obtained results are also presented here. We coded the approaches in C++ on a Linux platform with Ubuntu Desktop 20.04 LTS, running on an Intel(R) Core i5 6600 3.3 GHz with 8 GB RAM. Our implementations of the BPC algorithms are based on an *ad hoc branch-and-bound scheme which uses IBM ILOG CPLEX 12.10 as LP solver*.

6.1. Test instances

To evaluate the performances of the approaches, we have created two families of instances. It is quite accepted that stars in the sky are located like random points on a plane; this should not be confused with the fact each star has a light intensity and therefore stars in the real universe also show clustering. For that reason, there is no loss of generality by using random data for evaluating our implementations rather than using real-world catalogues of stars. The first family contains 240 instances generated to simulate real-world data from the telescope application motivating our research, where the bars move “on a line” and therefore the transition times depend on *one* coordinate of the objects. The transition time in the second family involves the two coordinates of objects, as one could expect in a vehicle routing context arising “on a plane”. To measure the impact of the synchronization, each family is divided in four classes according to the number of bands (processors) required to observe (process) each target (job). This section details how the instances were generated.

Our first step was to build five catalogues, each one containing of 500 targets randomly-generated in the 240.0×398.4 square arcseconds field of view of EMIR. We characterize each target i by a tuple (u_i, v_i, p_i, b_i) . The horizontal coordinate value u_i and the vertical value v_i have been randomly selected in the continuous intervals $[-120.0, 120.0]$ and $[-199.2, 199.2]$, respectively. The processing time p_i is an integer number uniformly generated in the interval $[1, 200]$. The value of the prize b_i has been randomly selected in the set $\{1, 10, 100\}$. We have characterized the initial and final configurations (i.e. the dummy targets 0 and $n+1$) by the tuple $(0, 0, 0, 0)$. We do not use each catalogue as one instance but a source of instances for the two families.

Aiming at a realistic design, the transition times in the first family of instances are defined as the horizontal movement of the sliding bar, i.e. $d_{ij} = |u_i - u_j|$. The transition times in the second family of instances are defined by the Euclidean distance between the points, i.e. $d_{ij} = \sqrt{(u_i - u_j)^2 + (v_i - v_j)^2}$. The first family is denoted by $\mathbb{H}$ and the second family by $\mathbb{E}$.

We consider $|K| = 55$ pairs of bars, each one k covering the targets in position (u_i, v_i) with $398.4 \times (k-1)/55 - 199.2 \leq v_i < 398.4 \times k/55 - 199.2$ for $k = 1, \dots, 55$. We create V^k for each instance and band $k \in K$ according to four classes. The instances in Class g (for $g \in \{1, 2, 3, 4\}$) have g bands assigned to each target, i.e. $|K_j| = g$ for all $j \in V \setminus \{0, n+1\}$. The bands assigned to a target in position (u_i, v_i) are selected minimizing the distance v_i to the center of the band. By doing so, the instances in Class 1 do not require synchronization. Hence, these instances help to analyze the behaviour of our algorithms on the problem without synchronization, which in a sense is similar to the classical OP. On the other hand, the synchronization requirements has a larger impact on the instances in Class 4, which are the most difficult to solve. The instances in the telescope application belong to Family $\mathbb{H}$ and combine characteristics of the four classes.

For each catalogue of targets, we have generated instances with different sizes. We have chosen these sizes according to a preliminary computational experience, aiming at evaluating the behavior of the algorithms ranging from easy to hard instances. Thus, the number n of targets for Class 1 is in the set $\{425, 450, 475, 500\}$, for Class 2 in the set $\{100, 110, 120, 130\}$, for Class 3 in the set $\{55, 60, 65, 70\}$, and for Class 4 in the set $\{45, 50, 55, 60\}$. The n targets for each instances are randomly selected from the 500 targets of the catalogue.

As done in Fischetti et al. (1998) for the OP, we define the time limitation for each instance of the family $\mathbb{E}$ as $L = \lceil \alpha \cdot \max\{z^{TSP}(V^k) : k \in K\} \rceil$, where α is a given parameter in $\{0.25, 0.50, 0.75\}$ and $z^{TSP}(V^k)$ is the length of the optimal TSP route on V^k . The time limitation for each instance of the Family $\mathbb{H}$ is defined by $L = \lceil 2 \cdot \alpha \cdot (\max\{u_j \mid j \in V\} - \min\{u_j \mid j \in V\}) \rceil$ for $\alpha \in \{0.25, 0.50, 0.75\}$.

Summarizing, our benchmark consists of eight groups of instances,

denoted with a letter (H or E) for the family and a number (1, 2, 3, 4) for the class. Each group contains a number of instances equal to five (one from each catalogue) times four values of n and times three values of α , i.e. a group consists of 60 SRPS instances. The benchmark, as well as its description and generation code, are available as a supplementary material in this publication.⁴

6.2. Tables description

We have conducted experiments solving the above instances with the approaches described in this paper. Details are shown on five tables. Each line of a table shows the average results of solving the five instances (one from each catalogue) for a given family, class, n and α . We have established a time limit of one CPU hour to solve each instance. When this time limit has been elapsed, the execution of the algorithm stops and the quality of the integer gap and the number of solved branching nodes are measured to get an estimation on the future behavior of the algorithm.

Table 1 shows the behavior of the software IBM ILOG CPLEX 12.10 solving the compact model on the first family of instances described in Section 6.1, i.e. the instances with distances on a line. Note that the commercial software uses sophisticated primal heuristic procedures, strong branching, general-purpose cuts, and pre-solving procedures, and we used the default settings. The first four columns show: n , number of targets; $|V^k|$, average number of targets associated to a band (not including the two dummy nodes); value of the parameter α ; and $|V^*|$, average number of targets observed in a feasible solution (possibly optimal) for these input instances. We also show the average computational time in CPU seconds to process the root node of the branching tree (Column RT), and to process the overall algorithm (Column OA). Column Gap (%) shows the average integer gap at the end of the root node (RT) and at the end of the execution (OA). We compute the integer gap as $100 \times (B_d - B_p) / B_p$, where B_d and B_p are the values of the dual and primal bounds, respectively. The primal bound is the objective value of the best integer feasible solution, and the dual bound is the optimal value of the best LP relaxation. The last columns of the tables show the number of branching nodes (#nod), and the number of instances solved to optimality within the time limit of one hour (✓).

Tables 2–5 summarize the result of the experiments showing details on the BPC algorithms. These tables include the previous columns, and also the number of variables generated by the BPC algorithms (#col), the number of Inequalities (17), the number of Inequalities (18), and number of branching nodes (#nod). Note that #col shows the number generated λ_r^k variables, and to get the final number of columns in the model one must add the number of x_{ij}^k and y_i variables. In a similar way, (17) and (18) show the number of generated inequalities, and to get the final number of rows in the model one must add the number of Eqs. (4)–(7), (19) and (21).

6.3. Discussion

Table 1 confirms the drawback of having a big- M value in the model SRPS-1 to activate/deactivate the conditional constraints associated with the synchronization. Indeed, the dual bounds obtained during the LP-relaxation become extremely weak, leading to an endless branching process. Those instances showing an important impact of the time limitation constraints yield a large integer gap at the root node. This is the case of the instances with $\alpha = 0.25$, showing integer gaps, in some cases (H2 with $n = 130$) of about 46% on average. However, even with a better integer gap at the end of the root node, the instances involving larger values of the time limitation L became harder. Indeed, only a few of these instances were solved to optimality. We do not show results

solving instances E through model SRPS-1 because they are very similar to the ones on the instances H.

The BPC algorithms reach tighter bounds and therefore better integer gaps. The better behaviour of these decomposition approaches versus solving the compact model is quite obvious on Group H1. Tables 2–5 show very few cases reaching gaps of over 1% at the end of the root node. All cases have finished the computation of the root node in less than a second, so the BPC algorithms used as an heuristic approach on these instances provide solutions with good features, which is a positive result for the telescope application motivating our research. Proving optimality seems more sensitive as, over the 5 instances in each row of a table, either a problem is solved in a few seconds or goes to the time limit of one hour.

It is common to observe better results of a column-generation algorithm when generating non-elementary routes rather than elementary routes (see e.g. Martinelli et al. (2014)). Indeed, maintenance of the labels for the latter jeopardizes the overall computational time. Also because the pricing problem for non-elementary routes is computational easier than the pricing problem for elementary routes. However, we note in our computational experiment that the use of the non-elementary pricing algorithm does not show any advantage when compared with the elementary version. The small size of the maximum number of targets (column $|V^k|$) requiring a processor (ranging from 6 to 17 targets on average, modulated by the parameter α) may explain this behavior. The elementary pricing algorithm provides tighter dual bounds. This observation is quite evident observing the differences the integer gap at the root node between algorithms BPC-OPS-E and BPC-OPS-N. In general, however, the two BPC algorithms have similar behaviours, and the computational time consumed by the pricing process almost reaches 50% of the overall computing time on average.

Although column $|V^k|$ shows that the routes of the processors are small, they are not independent routes because selecting a customer j to be observed forces all routes of processors in K_j to visit that customer.

Another interesting observation is about the number of generated columns. For most of the instances, it is close to 1000, which is a small number. This is uncommon when using column-generation approaches, and we believe it is a consequence of the relatively small values $|V^k|$ in our instances.

When comparing the performance of the BPC algorithms on instances “on a line” (Tables 2 and 3) versus instances “on the plane” (Tables 4 and 5), the latter seems to be slightly more difficult, with the larger differences on instances with targets requiring three bands and large L (e.g., the algorithms solved to optimality 5 instances in Group H3 with $n = 60$ and $\alpha = 0.75$, and only 2 in group E3 with the same n and α).

Finally, most of the use of computational resources has to do with synchronization issues. The number of generated synchronization constraints (mostly Inequalities (17)) grows exponentially with the number of bands required for the observation of each target. Indeed, when there are objects requiring a larger number of bands to be observed, the constrained shortest path problems are more related, and solving the SRPS is harder. The synchronization requirement is more critical on these instances. It impacts negatively not only on the computational time, but also on the use of the computer memory (the number of saved branch-and-bound nodes is much larger).

7. Conclusions

We have introduced a new optimization problem arising in the scheduling of observation targets for a telescope instrument which can perform up to 55 observations simultaneously. It combines multiple instances of the OP and synchronization constraints. The problem is first modelled with a simple compact model making use of a big- M value. The weakness of the linear programming relaxation motivated the design of another formulation that makes use of column and row generations. The column generation is based on a subproblem solved with Dynamic

⁴ Also available at <https://github.com/RieraULL/OPS-Benchmark>.

Table 1
Compact model SRPS-1 solved by IBM ILOG CPLEX 12.10.

<i>n</i>	$ V_k $	α	$ V^* $	CPU (sec.)		Gap (%)		#nod	✓
				RT	OA	RT	OA		
Group H1									
425	16.6	0.25	224.4	1.0	1 h	31.84	21.44	1372237	0
		0.50	366.8	0.8	1 h	4.44	4.35	1825989	0
		0.75	414.8	0.7	1 h	0.13	0.13	4227632	0
450	17.0	0.25	231.8	1.0	1 h	39.88	33.38	1108651	0
		0.50	382.8	0.8	1 h	6.51	6.46	1871202	0
		0.75	438.6	0.7	1 h	0.25	0.25	3443596	0
475	17.2	0.25	242.3	1.2	1 h	38.68	32.57	883122	0
		0.50	401.4	0.9	1 h	6.13	6.08	1456411	0
		0.75	460.2	0.7	1 h	0.41	0.41	3303323	0
500	17.6	0.25	248.8	1.2	1 h	45.97	40.82	759719	0
		0.50	418.6	1.0	1 h	7.55	7.50	1358782	0
		0.75	482.2	0.8	1 h	0.69	0.69	2573756	0
Group H2									
100	9.2	0.25	47.0	0.5	732.4	19.14	1.29	425085	4
		0.50	77.8	0.4	2881.6	4.94	2.55	2802659	1
		0.75	94.6	0.3	1615.4	0.21	0.09	1725292	3
110	10.0	0.25	49.8	0.4	937.6	25.80	2.09	620185	4
		0.50	84.6	0.4	1 h	5.66	3.81	2302131	0
		0.75	103.8	0.4	1 h	1.35	1.29	3891895	0
120	11.0	0.25	54.4	0.5	2331.0	35.24	12.57	1096238	2
		0.50	94.2	0.4	1 h	6.82	4.95	3580850	0
		0.75	112.4	0.4	1 h	0.65	0.57	3299686	0
130	11.4	0.25	55.6	0.5	2929.6	46.69	16.90	1194113	1
		0.50	99.8	0.5	1 h	6.96	5.31	1979459	0
		0.75	120.0	0.4	1 h	2.04	1.97	3944808	0
Group H3									
55	6.6	0.25	22.2	0.1	2.8	13.90	0.00	3570	5
		0.50	38.2	0.3	724.8	7.48	1.45	219762	4
		0.75	50.0	0.3	1457.9	0.58	0.15	1184713	3
60	7.6	0.25	24.4	0.3	9.0	13.19	0.00	11315	5
		0.50	45.6	0.3	1444.2	5.51	1.27	871416	3
		0.75	56.6	0.2	944.9	0.43	0.11	1066224	4
65	8.6	0.25	26.2	0.4	9.8	16.29	0.00	8541	5
		0.50	48.4	0.4	2306.3	6.45	3.28	1194224	2
		0.75	60.4	0.3	1573.4	1.39	1.11	1379223	3
70	9.0	0.25	30.4	0.5	638.3	24.94	0.00	219174	5
		0.50	53.4	0.4	1 h	5.18	2.05	1895638	0
		0.75	65.0	0.3	3311.9	1.31	1.07	2767077	1
Group H4									
45	7.0	0.25	17.2	0.2	1.9	8.86	0.00	1756	5
		0.50	30.2	0.4	792.6	9.15	0.04	556192	4
		0.75	39.4	0.3	986.9	0.76	0.02	608451	4
50	7.4	0.25	18.4	0.3	10.6	19.31	0.00	9404	5
		0.50	32.8	0.4	1786.7	10.37	2.00	649354	3
		0.75	42.4	0.4	2922.2	1.19	0.19	1987362	1
55	8.0	0.25	20.4	0.3	792.7	34.04	0.00	559349	5
		0.50	35.0	0.4	1 h	16.22	8.95	1223472	0
		0.75	48.0	0.4	1 h	2.51	1.84	2149506	0
60	9.2	0.25	23.6	0.4	1713.2	33.53	7.23	770551	3
		0.50	39.8	0.4	1 h	10.67	8.79	1339406	0
		0.75	53.0	0.4	1 h	6.62	6.57	2532200	0

Programming to manage the length requirement of the OP instances. The row generation separates combinatorial cuts to forbid sequences of observation targets needing bars that cannot be synchronized.

We have implemented two exact BPC algorithms based on the alternative formulation. The first one performs its column generation considering elementary routes, while the second one also allows non-elementary routes. We have designed computational experiments that simulate some scenarios involving a different number of targets for the telescope instrument and varying the level of synchronization. The use of combinatorial cuts to perform the synchronization constraints has proven to be an effective alternative to a compact model.

We have studied the performance of the algorithms on two families

of instances, one with distances on a line as in the telescope application, and another with Euclidean distances as it could occur in a vehicle routing application.

Although the pricing subproblem in the second BPC algorithm generates columns faster because it performs a non-elementary path generation, the low quality of the produced dual bound weakens the performance of the overall algorithm, and therefore shows slightly worse performance. However, the first BPC algorithm has been able to solve to optimality, within a time limit of one hour, instances with up to 500 objects under a low level of synchronization and up to 60 objects under the requirement of four processors for each object.

We believe that the introduced problem may appear in other vehicle

Table 2
Algorithms BPC-SRPS-E and BPC-SRPS-N on instances H1 and H2.

				BPC-SRPS-E								BPC-SRPS-N									
				CPU (sec.)		Gap (%)						CPU (sec.)		Gap (%)							
<i>n</i>	$ V_k $	α	$ V^* $	RT	OA	RT	OA	#col	(17)	(18)	#nod	✓	RT	OA	RT	OA	#col	(17)	(18)	#nod	✓
Group H1																					
425	16.6	0.25	230.2	1.1	1.1	0.00	0.00	1769	0	0	1	5	1.5	1.5	0.00	0.00	1465	0	0	1	5
		0.50	369.2	24.0	24.0	0.00	0.00	1708	0	0	1	5	35.7	35.7	0.00	0.00	1538	0	0	1	5
		0.75	416.0	51.7	51.7	0.00	0.00	1452	0	0	1	5	56.3	56.3	0.00	0.00	1255	0	0	1	5
450	17.0	0.25	239.2	1.8	1.8	0.00	0.00	1904	0	0	1	5	2.2	2.2	0.00	0.00	1746	0	0	1	5
		0.50	390.0	64.7	64.7	0.00	0.00	1897	0	0	1	5	47.1	47.1	0.00	0.00	1718	0	0	1	5
		0.75	439.8	73.0	73.0	0.00	0.00	1494	0	0	1	5	64.3	64.3	0.00	0.00	1477	0	0	1	5
475	17.2	0.25	246.0	1.8	1.8	0.00	0.00	2019	0	0	1	5	2.3	2.3	0.00	0.00	1892	0	0	1	5
		0.50	405.8	44.2	44.2	0.00	0.00	2018	0	0	1	5	55.8	55.8	0.00	0.00	1964	0	0	1	5
		0.75	464.0	83.4	83.5	0.00	0.00	1586	0	0	1	5	80.5	80.5	0.00	0.00	1564	0	0	1	5
500	17.6	0.25	261.6	4.2	4.2	0.00	0.00	2231	0	0	1	5	4.4	4.4	0.00	0.00	2060	0	0	1	5
		0.50	425.2	129.1	129.1	0.00	0.00	2116	0	0	1	5	132.9	132.9	0.00	0.00	2078	0	0	1	5
		0.75	486.0	207.2	207.2	0.00	0.00	1641	0	0	1	5	216.2	216.2	0.00	0.00	1731	0	0	1	5
Group H2																					
100	9.2	0.25	47.0	0.0	0.0	0.00	0.00	451	1	0	1	4	0.0	0.1	0.41	0.00	358	53	0	5	4
		0.50	77.8	0.0	0.5	0.00	0.00	735	119	4	17	5	0.1	0.7	0.05	0.00	548	63	0	28	5
		0.75	94.6	0.0	0.0	0.00	0.00	748	0	0	1	5	0.1	0.1	0.00	0.00	588	0	0	1	3
110	10.0	0.25	49.8	0.0	0.2	0.29	0.00	518	34	16	10	4	0.0	0.4	0.67	0.00	407	2	36	20	5
		0.50	85.2	0.1	0.8	0.00	0.00	868	266	0	33	5	0.1	4.7	0.47	0.00	681	4	0	148	5
		0.75	103.6	0.1	1.1	0.00	0.00	861	265	0	17	5	0.2	1.6	0.03	0.00	733	15	0	33	3
120	11.0	0.25	54.6	0.1	721.0	0.68	0.07	634	64951	36540	24565	4	0.1	721.4	1.38	0.07	510	62990	49093	23373	4
		0.50	95.0	0.1	2.5	0.00	0.00	980	401	33	49	5	0.2	723.2	0.22	0.01	876	40511	478	4942	3
		0.75	113.6	0.1	4.2	0.00	0.00	983	628	0	65	5	0.4	14.0	0.05	0.00	1028	121	0	65	4
130	11.4	0.25	57.2	0.1	720.6	0.68	0.01	707	45175	30097	31409	4	0.1	722.9	0.76	0.07	563	57353	39048	23997	4
		0.50	100.8	0.1	1445.3	0.06	0.06	1104	146873	1788	17124	3	0.2	1450.0	0.32	0.09	1068	129671	1419	15976	3
		0.75	121.6	0.2	1449.9	0.61	0.61	1087	33251	0	3302	3	0.7	1453.3	0.18	0.14	1289	13288	0	3046	3

Table 3
Algorithms BPC-SRPS-E and BPC-SRP-N on instances H3 and H4.

				BPC-SRPS-E									BPC-SRPS-N								
				CPU (sec.)		Gap (%)		#col	(17)	(18)	#nod	✓	CPU (sec.)		Gap (%)		#col	(17)	(18)	#nod	✓
n	V _k	α	V [*]	RT	OA	RT	OA						RT	OA	RT	OA					
Group H3																					
55	6.6	0.25	22.2	0.0	0.0	0.00	0.00	271	0	0	1	5	0.0	0.0	0.00	0.00	228	1270	0	1	5
		0.50	38.2	0.0	1.4	0.01	0.00	529	335	199	119	5	0.0	4.7	0.17	0.00	423	1271	468	373	5
		0.75	50.0	0.0	0.0	0.00	0.00	575	0	0	1	5	0.0	0.1	0.04	0.00	438	0	0	3	5
60	7.6	0.25	24.4	0.0	0.0	0.00	0.00	352	1	0	1	5	0.0	0.0	0.00	0.00	288	1	0	1	5
		0.50	45.8	0.0	0.4	0.00	0.00	631	152	10	17	5	0.0	1.0	0.37	0.00	493	104	0	41	5
		0.75	56.8	0.0	0.0	0.00	0.00	646	0	0	1	5	0.1	0.1	0.09	0.00	552	1	0	3	5
65	8.6	0.25	26.2	0.0	0.0	0.00	0.00	407	2	0	1	5	0.0	0.0	0.00	0.00	335	3	0	1	5
		0.50	48.6	0.0	720.9	0.01	0.01	708	108710	2384	11136	4	0.1	720.7	0.59	0.01	579	105331	2106	11266	4
		0.75	60.6	0.0	1.2	0.00	0.00	730	434	0	33	5	0.1	1.5	0.01	0.00	653	152	0	33	5
70	9.0	0.25	30.4	0.0	0.1	0.06	0.00	500	17	6	5	5	0.0	0.2	0.60	0.00	394	12	1	12	5
		0.50	53.2	0.1	721.9	0.08	0.08	815	123735	16211	18205	4	0.1	722.6	0.13	0.08	695	123492	16203	18207	4
		0.75	65.0	0.1	1.6	0.00	0.00	817	512	0	33	5	0.2	1.8	0.02	0.00	822	92	0	33	5
Group H4																					
45	7.0	0.25	17.2	0.0	0.0	0.00	0.00	316	0	0	1	5	0.0	0.0	0.00	0.00	257	0	0	1	5
		0.50	30.2	0.0	0.0	0.00	0.00	607	0	0	1	5	0.0	0.0	0.00	0.00	437	0	0	1	5
		0.75	39.4	0.0	0.0	0.00	0.00	671	0	0	1	5	0.0	0.0	0.00	0.00	487	0	0	1	5
50	7.4	0.25	18.4	0.0	0.0	0.00	0.00	368	0	0	1	5	0.0	0.0	0.00	0.00	298	0	0	1	5
		0.50	33.0	0.0	0.0	0.00	0.00	691	0	0	1	5	0.1	0.1	0.06	0.00	538	2	0	1	5
		0.75	42.4	0.0	0.6	0.00	0.00	762	244	0	17	5	0.1	0.9	0.00	0.00	701	231	0	17	5
55	8.0	0.25	20.4	0.0	0.0	0.00	0.00	440	0	0	1	5	0.0	0.2	0.00	0.00	359	42	8	6	5
		0.50	36.0	0.1	4.0	0.01	0.00	823	2180	13	145	5	0.1	2.7	0.12	0.00	698	974	3	81	5
		0.75	48.2	0.1	6.3	0.00	0.00	871	1149	0	49	5	0.2	30.6	0.02	0.00	952	973	0	113	5
60	9.2	0.25	23.8	0.0	0.3	0.04	0.00	569	81	20	14	5	0.0	0.4	0.12	0.00	454	221	2	19	5
		0.50	40.6	0.1	1378.3	0.12	0.11	1006	322372	1056	26724	3	0.1	1441.9	0.17	0.11	943	312922	498	25099	3
		0.75	54.4	0.1	730.7	0.12	0.12	980	28889	0	1873	4	0.3	808.6	0.14	0.04	1223	30372	0	2002	4

Table 4
Algorithms BPC-SRPS-E and BPC-SRPS-N on instances E1 and E2.

				BPC-SRPS-E								BPC-SRPS-N									
				CPU (sec.)		Gap (%)						CPU (sec.)		Gap (%)							
n	$ V_k $	α	$ V^* $	RT	OA	RT	OA	#col	(17)	(18)	#nod	✓	RT	OA	RT	OA	#col	(17)	(18)	#nod	✓
Group E1																					
425	16.6	0.25	206.6	2.2	2.2	0.00	0.00	1642	0	0	1	5	1.3	1.3	0.00	0.00	1336	0	0	1	5
		0.50	359.2	60.4	60.4	0.00	0.00	1745	0	0	1	5	45.7	45.7	0.00	0.00	1588	0	0	1	5
		0.75	414.2	67.4	67.5	0.00	0.00	1473	0	0	1	5	62.9	62.9	0.00	0.00	1325	0	0	1	5
450	17.0	0.25	220.4	2.4	2.4	0.00	0.00	1746	0	0	1	5	2.3	2.3	0.00	0.00	1584	0	0	1	5
		0.50	376.6	54.7	54.8	0.00	0.00	1905	0	0	1	5	75.8	75.8	0.00	0.00	1772	0	0	1	5
		0.75	437.4	101.3	101.3	0.00	0.00	1566	0	0	1	5	72.1	72.1	0.00	0.00	1472	0	0	1	5
475	17.2	0.25	227.0	2.0	2.0	0.00	0.00	1896	0	0	1	5	2.1	2.1	0.00	0.00	1721	0	0	1	5
		0.50	391.0	41.9	41.9	0.00	0.00	1999	0	0	1	5	67.9	67.9	0.00	0.00	2051	0	0	1	5
		0.75	460.6	75.5	75.5	0.00	0.00	1600	0	0	1	5	75.9	75.9	0.00	0.00	1604	0	0	1	5
500	17.6	0.25	241.0	3.6	3.6	0.00	0.00	2099	0	0	1	5	5.8	5.8	0.00	0.00	1880	0	0	1	5
		0.50	411.6	120.6	120.7	0.00	0.00	2131	0	0	1	5	123.2	123.2	0.00	0.00	2094	0	0	1	5
		0.75	482.0	166.3	166.4	0.00	0.00	1601	0	0	1	5	181.0	181.0	0.00	0.00	1813	0	0	1	5
Group E2																					
100	9.2	0.25	36.8	0.0	0.0	0.0	0.0	321	0	0	1	5	0.0	0.1	0.5	0.0	269	140719	0	10	5
		0.50	72.2	0.0	1156.1	0.0	0.0	713	83263	34440	27253	4	0.1	1156.0	0.0	0.0	544	110679	37733	31919	4
		0.75	92.2	0.0	1.4	0.0	0.0	738	136	6	17	5	0.1	1.1	0.0	0.0	592	52	0	18	5
110	10.0	0.25	39.0	0.0	0.4	0.0	0.0	376	33	76	36	5	0.0	0.2	0.0	0.0	314	83272	44	17	5
		0.50	77.4	0.1	2162.4	0.1	0.1	842	236184	103648	61147	2	0.1	2162.1	0.6	0.1	685	223909	99624	59815	2
		0.75	101.6	0.1	1.8	0.0	0.0	872	315	8	33	5	0.1	2.5	0.0	0.0	763	79	0	33	5
120	11.0	0.25	42.8	0.0	2.5	0.2	0.0	464	121	342	201	5	0.1	3.0	0.7	0.0	391	225	328	232	5
		0.50	86.8	0.1	1445.7	0.1	0.1	941	141586	24175	22485	3	0.1	1445.0	0.3	0.1	871	120528	23996	19160	3
		0.75	111.0	0.1	29.7	0.0	0.0	1006	1191	9	129	5	0.5	24.9	0.0	0.0	1003	397	0	113	5
130	11.4	0.25	45.8	0.1	0.8	0.1	0.0	526	83	78	47	5	0.1	1.9	2.9	0.0	425	81	69	152	5
		0.50	92.4	0.1	2163.8	0.2	0.2	1084	241228	41399	42461	2	0.2	2170.2	0.7	0.2	1029	200187	39595	34222	2
		0.75	117.8	0.2	1504.6	0.2	0.2	1136	54437	19	5622	3	0.6	1453.7	0.2	0.2	1252	19674	0	4237	3

Table 5
Algorithms BPC-SRPS-E and BPC-SRPS-N on instances E3 and E4.

				BPC-SRPS-E									BPC-SRPS-N								
n	V _k	α	V [*]	CPU (sec.)		Gap (%)		#col	(17)	(18)	#nod	✓	CPU (sec.)		Gap (%)		#col	(17)	(18)	#nod	✓
				RT	OA	RT	OA						RT	OA	RT	OA					
Group E3																					
55	6.6	0.25	15.6	0.0	0.0	0.00	0.00	181	0	0	1	5	0.0	0.0	0.01	0.01	160	96114	0	1	5
		0.50	34.6	0.0	293.4	0.01	0.00	479	92188	16040	19253	5	0.0	298.5	0.02	0.00	356	95994	13569	18943	5
		0.75	48.0	0.0	0.4	0.00	0.00	571	238	0	17	4	0.1	0.1	0.04	0.01	460	6	0	2	4
60	7.6	0.25	24.6	0.0	0.0	0.00	0.00	315	0	0	1	5	0.0	0.0	0.02	0.02	208	0	0	1	5
		0.50	44.2	0.0	0.3	0.01	0.01	620	251771	1535	21625	5	0.0	0.5	0.37	0.00	461	234452	0	20	5
		0.75	42.0	0.0	1889.5	0.01	0.01	494	228775	902	19715	2	0.1	1889.7	0.07	0.04	603	485942	3854	62995	1
65	8.6	0.25	45.0	0.0	1378.5	0.17	0.11	674	382850	48397	47564	3	0.0	0.0	0.02	0.02	245	2	0	1	5
		0.50	58.6	0.0	1313.5	0.02	0.02	744	355333	8735	33387	5	0.0	1441.7	0.74	0.11	553	392324	43570	47989	3
		0.75	21.8	0.0	0.0	0.00	0.00	297	2	0	1	3	0.1	1313.8	0.03	0.02	647	386446	4459	32900	3
70	9.0	0.25	50.0	0.1	1441.5	0.10	0.10	784	347230	20966	43153	4	0.0	60.6	0.67	0.02	306	6921	8219	4233	5
		0.50	63.2	0.1	537.6	0.01	0.01	841	228635	59	17194	4	0.1	1441.8	0.15	0.10	638	310711	22466	39915	3
		0.75	24.6	0.0	52.6	0.07	0.00	376	7905	6806	3803	4	0.2	537.9	0.02	0.01	777	228113	59	17178	4
Group E4																					
45	7.0	0.25	11.8	0.0	0.0	0.00	0.00	212	0	0	1	5	0.0	0.0	0.00	0.00	182	268344	0	1	5
		0.50	27.4	0.0	0.0	0.00	0.00	565	0	0	1	5	0.0	0.0	0.00	0.00	414	1	0	1	5
		0.75	38.6	0.0	0.0	0.00	0.00	663	0	0	1	5	0.0	0.0	0.00	0.00	465	0	0	1	5
50	7.4	0.25	13.6	0.0	0.1	0.02	0.00	245	19	1	3	5	0.0	0.1	0.02	0.00	212	457935	1	2	5
		0.50	29.8	0.1	2152.7	0.16	0.14	668	763800	11078	69775	3	0.1	2176.4	0.49	0.14	525	726263	17990	69133	3
		0.75	41.2	0.0	0.0	0.00	0.00	737	0	0	1	5	0.1	0.1	0.05	0.00	667	17	0	4	5
55	8.0	0.25	16.0	0.0	1.0	0.02	0.00	304	443	45	83	5	0.0	1.2	0.02	0.00	261	494	42	92	5
		0.50	33.0	0.1	1443.6	0.12	0.12	765	381420	43569	56451	2	0.1	1444.5	0.72	0.12	648	347938	64204	54033	2
		0.75	44.8	0.1	2166.9	0.11	0.11	898	298005	1	18942	3	0.1	2194.7	0.40	0.31	948	288261	0	18495	2
60	9.2	0.25	18.8	0.0	0.1	0.02	0.00	414	13	7	4	5	0.0	0.1	0.02	0.00	345	17	7	4	5
		0.50	38.0	0.1	2164.7	0.44	0.44	956	611024	9686	53564	4	0.1	2162.7	0.53	0.44	844	658463	4500	52370	4
		0.75	51.4	0.1	2864.3	1.64	1.64	958	363250	0	18561	1	0.2	2890.9	1.53	1.49	1190	323703	0	20155	1

routing contexts like those related to the manpower allocation problem (mentioned in Section 3.3) where not all tasks are mandatory but some must be selected to maximize the benefit.

A challenging question for future research is to investigate new branching procedures, perhaps involving the time variables s , and to investigate the use of traditional Benders' cuts when the time variables s are projected out. Also the Appendix in this manuscript suggests measuring the impact of solving directly $SP_{\alpha\beta\gamma\delta}^k$ rather than $SP_{\alpha\beta}^k$ when generating the columns for the set-partitioning model.

CRedit authorship contribution statement

Jorge Riera-Ledesma: Conceptualization, Software, Validation.
Juan-José Salazar-González: Conceptualization, Methodology, Writing - original draft, Writing - review & editing, Supervision, Funding acquisition.

Appendix A. Linking the three models

This section shows how models SPR-2 and SPR-3 can be mathematically derived from model SPR-1. To this end, we need new variables y_j^k and s_j^k in addition to the x_{ij}^k for each $k \in K$. Variable y_j^k indicates whether job j is processed or not on the selected path by k . Variable s_j^k represents the time when j starts being processed if $y_j^k = 1$, and it assumes value 0, otherwise. We use $s_0^k = 0$ and $s_{n+1}^k = L$ without loss of generality. Then an alternative compact model for SPRs, called SPR-1', is:

$$\max \sum_{j \in J} b_j y_j \quad (A.1)$$

subject to

$$x^k(\delta_+^k(0)) = 1 \quad k \in K \quad (A.2)$$

$$x^k(\delta_-^k(n+1)) = 1 \quad k \in K \quad (A.3)$$

$$x^k(\delta_+^k(j)) - x^k(\delta_-^k(j)) = 0 \quad k \in K, j \in J^k \quad (A.4)$$

$$x^k(\delta_-^k(j)) = y_j^k \quad k \in K, j \in J^k \quad (A.5)$$

$$0 \leq s_j^k \leq L y_j^k \quad k \in K, j \in J^k \quad (A.6)$$

$$s_i^k - s_j^k + L x_{ij}^k \leq L - t_{ij} \quad k \in K, (i, j) \in A^k \quad (A.7)$$

$$s_{n+1}^k - s_0^k \leq L \quad k \in K \quad (A.8)$$

$$x_{ij}^k \in \{0, 1\} \quad k \in K, (i, j) \in A^k \quad (A.9)$$

$$y_j^k \in \{0, 1\} \quad k \in K, j \in J^k \quad (A.10)$$

$$y_j = y_j^k \quad k \in K, j \in J^k \quad (A.11)$$

$$s_j = s_j^k \quad k \in K, j \in J^k. \quad (A.12)$$

Model SPR-1' reduces to Model SPR-1 when eliminating the variables y_j^k and s_j^k with (A.11) and (A.12), respectively. Model SPR-1' suggests using Dantzig-Wolfe Decomposition by reformulating the domain D^k defined by (A.2)–(A.10) for each k . Such a domain comprises.

(A.2)–(A.4): the path constraints from node 0 to node $n+1$;

(A.5)–(A.6): the processed jobs selected from J^k and a zero time value for the unprocessed ones;

(A.7): the expressions linking the flow variables x_{ij}^k and the time variables s_i^k and s_j^k satisfying the precedence constraints on jobs i and j processed by k ;

(A.8): the horizon length;

(A.9)–(A.10): the binary requirements on the x^k and y^k variables.

The domain D^k is a bounded polyhedron and an extreme point of the convex hull of D^k is a path $r \in \Omega^k$. For each path $r \in \Omega^k$, in coherence with notation in Section 4.3, let us denote by.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

The authors thank Prof. Francisco Garzón López at the “Instituto de Astrofísica de Canarias” for posing and describing this optimization problem. This research has been partially supported by the Spanish projects MTM2015-63680-R (MINECO/FEDER, UE), PID2019-104928RB-I00 (MINECO/FEDER, UE) and ProID2017010132 (Gobierno de Canarias/FEDER, UE).

- $x_{ij,r}^k$ the characteristic vector being 1 if and only if $(i,j) \in A^k$ is in r ,
- $y_{j,r}^k$ the characteristic vector being 1 if and only if $j \in J^k$ is processed along r , and
- $s_{ij,r}^k$ a non-negative number representing the time to process k along r .

Any solution (x^k, y^k, s^k) in the convex hull of D^k can be written as

$$\sum_{r \in \Omega^k} \lambda_r^k = 1 \quad (A.13)$$

$$\lambda_r^k \geq 0 \quad r \in \Omega^k \quad (A.14)$$

$$\sum_{r \in \Omega^k} x_{ij,r}^k \lambda_r^k = x_{ij}^k \quad (i,j) \in \Omega^k \quad (A.15)$$

$$\sum_{r \in \Omega^k} y_{j,r}^k \lambda_r^k = y_j^k \quad j \in J^k \quad (A.16)$$

$$\sum_{r \in \Omega^k} s_{ij,r}^k \lambda_r^k = s_j^k \quad j \in J^k. \quad (A.17)$$

The substitution of the above consideration in Model SPR-1' gives the Dantzig-Wolfe master problem described by $DW_{\alpha\beta\gamma\pi}$ by (A.1) subject to (A.9)–(A.17) for all $k \in K$. For processor $k \in K$, the reduced cost $\bar{c}_r^k$ of variable λ_r^k for each $r \in \Omega^k$ is

$$\bar{c}_r^k = 0 - \left(\alpha^k + \sum_{(i,j) \in A^k} x_{ij,r}^k \beta_{ij}^k + \sum_{j \in J^k} y_{j,r}^k \pi_j^k + \sum_{j \in J^k} s_{j,r}^k \sigma_j^k \right),$$

where β_{ij}^k , π_j^k and σ_j^k are the dual variables of (A.15)–(A.17), respectively, of the linear relaxation of $DW_{\alpha\beta\gamma\pi}$. This means that finding the *maximum* reduced cost for processor k is equivalent to solve the following minimization subproblem, denoted by $SP_{\alpha\beta\gamma\pi}^k$:

$$\alpha^k + \min \sum_{(i,j) \in A^k} x_{ij,r}^k \beta_{ij}^k + \sum_{j \in J^k} y_{j,r}^k \pi_j^k + \sum_{j \in J^k} s_{j,r}^k \sigma_j^k$$

subject to (A.2)–(A.10). This subproblem is an elementary time-constrained shortest-path problem with linear node costs (see e.g. Ioachim et al. (1998)). The objective function includes

- a fix cost α^k given by the master solution;
- the dual value β_{ij}^k as a first part of the cost on each arc $(i,j) \in A^k$ of the path;
- the dual value π_j^k as a second part of the cost on each arc entering node $j \in J^k$;
- the dual value σ_j^k is a cost on each node $j \in J^k$ to pull backward or push forward (depending whether σ_j^k is positive or negative) the value of s_j^k in the synchronization process. Constraints (A.6) assign zero time to unprocessed jobs, thus $s_j^k \sigma_j^k = 0$ and therefore no impact in the objective function for unprocessed jobs.

Another approach to deal with SPR-1' is to reformulate only the x-variables when using the Dantzig-Wolfe Decomposition. In this case, the master problem, denoted as $DW_{\alpha\beta}$, is modeled by (A.1) subject to (A.13)–(A.15) and (7)–(11). The subproblem, denoted as $SP_{\alpha\beta}^k$, is again the elementary time-constrained shortest-path problem $SP_{\alpha\beta\gamma\pi}^k$ but now with the simpler objective function

$$\alpha^k + \min \sum_{(i,j) \in A^k} x_{ij,r}^k \beta_{ij}^k.$$

Note that in this case, since the s-variables are not reformulated, the constraints $0 \leq s_j^k \leq L_j$ for all $j \in J$ are unnecessary. Model SPR-3 differs from the master problem $DW_{\alpha\beta}$ in the precedence constraints (8) and (9) of the processed jobs described by the set $\bar{A}$ for a binary solution $(\tilde{x}, \tilde{y})$. These constraints are replaced by the combinatorial cuts (17) and (18) in Model SPR-3 to have only binary variables.

References

- Ahuja, R.K., Magnanti, T.L., Orlin, J.B., 1993. Network Flows: Theory, Algorithms, and Applications, Prentice Hall, Upper Saddle River, NJ, USA, ISBN 9780136175490.
- Alba Martínez, M., Cordeau, J.-F., Dell'Amico, M., Iori, M., 2013. A branch-and-cut algorithm for the double traveling salesman problem with multiple stacks. *INFORMS J. Comput.* 25, 41–55. <https://doi.org/10.1287/ijoc.1110.0489>.
- Ascheuer, N., Fischetti, M., Götschel, M., 2001. Solving the asymmetric travelling salesman problem with time windows by branch-and-cut. *Math. Program.* 90, 475–506. <https://doi.org/10.1007/PL00011432>.
- Baldacci, R., Mingozzi, A., Roberti, R., 2011. New route relaxation and pricing strategies for the vehicle routing problem. *Oper. Res.* 59, 1269–1283. <https://doi.org/10.1287/opre.1110.0975>.
- Bélangier, N., Desaulniers, G., Soumis, F., Desrosiers, J., 2006. Periodic airline fleet assignment with time windows, spacing constraints, and time dependent revenues. *Eur. J. Oper. Res.* 175, 1754–1766. <https://doi.org/10.1016/j.ejor.2004.04.051>.
- Butt, S.E., Cavalier, T., 1994. A heuristic for the multiple tour maximum collection problem. *Comput. Oper. Res.* 21, 101–111. [https://doi.org/10.1016/0305-0548\(94\)90065-5](https://doi.org/10.1016/0305-0548(94)90065-5).
- Chao, I.-M., Golden, B.L., Wasil, E.A., 1996. The team orienteering problem. *Eur. J. Oper. Res.* 88, 464–474. [https://doi.org/10.1016/0377-2217\(94\)00289-4](https://doi.org/10.1016/0377-2217(94)00289-4).

- Cordeau, J.-F., 2006. A branch-and-cut algorithm for the dial-a-ride problem. *Oper. Res.* 54, 573–586. <https://doi.org/10.1287/opre.1060.0283>.
- Cortés, C.E., Matamala, M., Contardo, C., 2010. The pickup and delivery problem with transfers: Formulation and a branch-and-cut solution method. *Eur. J. Oper. Res.* 200, 711–724. <https://doi.org/10.1016/j.ejor.2009.01.022>.
- Dantzig, G., Wolfe, P., 1960. Decomposition principle for linear programs. *Oper. Res.* 8 (1), 101–111. <https://doi.org/10.1287/opre.8.1.101>.
- Desrosiers, J., Lübbecke, M.E., 2011. *Branch-Price-and-Cut Algorithms*, John Wiley & Sons Inc. <https://onlinelibrary.wiley.com/doi/abs/10.1002/9780470400531.eorms0118>.
- Dijkstra, E.W., 1959. A note on two problems in connexion with graphs. *Numerische Mathematik* 1, 269–271. <https://doi.org/10.1007/BF01386390>.
- Dohn, A., Kolind, E., Clausen, J., 2009. The manpower allocation problem with time windows and job-teaming constraints: A branch-and-price approach. *Comput. Oper. Res.* 36, 1145–1157. <https://doi.org/10.1016/j.cor.2007.12.011>.
- Drexel, M., 2012. Synchronization in Vehicle Routing —Survey of VRPs with Multiple Synchronization Constraints. *Transp. Sci.* 46, 297–316. <https://doi.org/10.1287/trsc.1110.0400>.
- Fischetti, M., Salazar-González, J.J., Toth, P., 1998. Solving the orienteering problem through branch-and-cut. *INFORMS J. Comput.* 10, 133–148. <https://doi.org/10.1287/ijoc.10.2.133>.
- Gelareh, S., Merzouki, R., McGinley, K., Murray, R., 2013. Scheduling of intelligent and autonomous vehicles under pairing/unpairing collaboration strategy in container terminals. *Transp. Res. C* 33, 1–21. <https://doi.org/10.1016/j.trc.2013.04.006>.
- Golden, B., Levy, L., Vohra, R., 1987. The orienteering problem. *Naval Res. Logist.* 34, 359–366. [https://doi.org/10.1002/1520-6750\(198706\)34:3<307::AID-NAV3220340302>3.0.CO;2-D](https://doi.org/10.1002/1520-6750(198706)34:3<307::AID-NAV3220340302>3.0.CO;2-D).
- Gouveia, L., Pires, J.M., 1999. The asymmetric travelling salesman problem and a reformulation of the Miller–Tucker–Zemlin constraints. *Eur. J. Oper. Res.* 112 (1), 134–146. [https://doi.org/10.1016/S0377-2217\(97\)00358-5](https://doi.org/10.1016/S0377-2217(97)00358-5).
- Gunawan, A., Lau, H.C., Vansteenwegen, P., 2016. Orienteering Problem: A survey of recent variants, solution approaches and applications. *Eur. J. Oper. Res.* 255, 315–332. <https://doi.org/10.1016/j.ejor.2016.04.059>.
- Ioachim, I., Gélinas, S., Soumis, F., Desrosiers, J., 1998. A dynamic programming algorithm for the shortest path problem with time windows and linear node costs. *Networks* 31, 193–204. [https://doi.org/10.1002/\(SICI\)1097-0037\(199805\)31:3<193::AID-NET6>3.0.CO;2-A](https://doi.org/10.1002/(SICI)1097-0037(199805)31:3<193::AID-NET6>3.0.CO;2-A).
- Ioachim, I., Desrosiers, J., Soumis, F., Bélanger, N., 1999. Fleet assignment and routing with schedule synchronization constraints. *Eur. J. Oper. Res.* 119, 75–90. [https://doi.org/10.1016/S0377-2217\(98\)00343-9](https://doi.org/10.1016/S0377-2217(98)00343-9).
- Irnich, S., Desaulniers, G., 2005. Shortest path problems with resource constraints. In: Desaulniers, G., Desrosiers, J., Solomon, M., (Eds.), *Column Generation*, Springer, Boston, MA, pp. 33–65, doi: 10.1007/0-387-25486-2_2.
- Irnich, S., Villeneuve, D., 2006. The shortest-path problem with resource constraints and k-Cycle elimination for $k \geq 3$. *INFORMS J. Comput.* 18, 391–406. <https://doi.org/10.1287/ijoc.1040.0117>.
- Desrosiers, J., Dumas, Y., Solomon, M.M., Soumis, F., 1995. Time constrained routing and scheduling. In: Ball, M., Magnanti, T., Monma, C., Nemhauser, G. (Eds.), *Network Routing*, vol. 8 of *Handbooks in Operations Research and Management Science*, Elsevier, 35–139, doi: 10.1016/S0927-0507(05)80106-9.
- Johnson, D.B., 1975. Finding all the elementary circuits of a directed graph. *SIAM J. Comput.* 4, 77–84. <https://doi.org/10.1137/0204007>.
- Li, Y., Lim, A., Rodrigues, B., 2005. Manpower allocation with time windows and job-teaming constraints. *Naval Res. Logist.* 52, 302–311. <https://doi.org/10.1002/nav.20075>.
- Lim, A., Rodrigues, B., Song, L., 2004. Manpower allocation with time windows. *J. Opl. Res. Soc.* 55, 1178–1186 <http://www.jstor.org/stable/4101889>.
- Luo, Z., Qin, H., Zhu, W., Lim, A., 2016. Branch-and-price-and-cut for the manpower routing problem with synchronization constraints. *Naval Res. Logist.* 63, 138–171. <https://doi.org/10.1002/nav.21683>.
- Martinelli, R., Pécin, D., Poggi, M., 2014. Efficient elementary and restricted non-elementary route pricing. *Eur. J. Oper. Res.* 239, 102–111. <https://doi.org/10.1016/j.ejor.2014.05.005>.
- Salazar-González, J.-J., 2021. Designing optimal masks for a multi-object spectrometer. *Omega* 103, in press, doi: 10.1016/j.omega.2021.102392.
- Tsiligirides, T., 1984. Heuristic methods applied to orienteering. *J. Opl. Res. Soc.* 35, 797–809. <https://doi.org/10.2307/2582629>.
- Vansteenwegen, P., Souffriau, W., Oudheusden, D., 2011. The orienteering problem: A survey. *Eur. J. Oper. Res.* 209, 1–10. <https://doi.org/10.1016/j.ejor.2010.03.045>.